

The Definitive C++ OOP Handbook

A complete, beginner-friendly walk from "what is a class?" to vtables, move semantics, CRTP, and production-grade OOP design — every concept defined the first time it's used, every chapter ending in a mini quiz. This edition merges and deduplicates three prior guides into one: the full 16-chapter course, a rapid-fire FAQ round cross-referenced against GeeksforGeeks/InterviewBit/Simplilearn/Edureka/MyGreatLearning/ScienceTechEasy/FinalRoundAI, and five complete practice designs. Targets **C++17**, with notes wherever earlier/later standards change the picture.

16 chapters, basics → production OOP

2-week & 4-week study plans

Mini quiz after every chapter

75+ interview Q&A, categorized + rapid-fire

5 full practice designs

Updated Sep 2026

HOW TO USE THIS HANDBOOK

1. New to OOP entirely? Start at Chapter 1 and don't skip the mini quizzes — each one gates the next chapter's assumptions.
2. Already know OOP in another language? Skim Ch.1–2, then slow down from Ch.5 onward (C++-specific mechanics).
3. Cramming for an interview this week? Read Ch.6–9 (polymorphism + internals + resource management) closely, then go straight to the Q&A bank (Ch.14) and Practice (Ch.15).
4. Every chapter follows the same shape:
Explanation → C++ Example → Internal Mechanics → Pitfalls → Best Practices → Mini Quiz.

PREREQUISITES

- Comfortable writing basic C++ (variables, functions, loops, `#include`, compiling a single file).
- No prior OOP knowledge assumed — Chapter 1 starts from "what is a class?"
- Pointers/references helpful but re-explained where first needed (Ch.2–3).
- A compiler supporting C++17 (GCC 8+, Clang 7+, MSVC 2017 15.7+) if you want to run the examples.

WHEN	FOCUS
2-week plan	Days 1–3: Ch.1–4 (classes, lifecycle, const, abstraction). Days 4–7: Ch.5–8 (inheritance, polymorphism, internals, RAII/smart pointers). Days 8–10: Ch.9–13 (advanced inheritance, operators, CRTP/PIMPL, tools, design principles). Days 11–14: Ch.14–16, daily timed Q&A drills + 1 mini design/day.
4-week plan	Week 1: Ch.1–4, redo every mini quiz until perfect. Week 2: Ch.5–8, hand-trace the vtable diagrams yourself. Week 3: Ch.9–13, implement each case study without looking at the answer first. Week 4: Ch.14–16 — full Q&A bank, all practice problems, 2 mock interviews using Ch.15's designs.

CONTENTS

01	Classes, Objects & Encapsulation	2	02	Constructors, Destructors & Object Lifecycle	5
03	this, Static Members & Const-Correctness	8	04	Abstraction & Interfaces	11
05	Inheritance Fundamentals	13	06	Polymorphism & Virtual Functions	16
07	Internals: vtable, Binding, Layout, Slicing & RTTI	19	08	Resource Management: Rule of 0/3/5, RAII, Smart Pointers	22
09	Advanced Inheritance & Composition	26	10	Operator Overloading	29
11	Compile-Time OOP: Templates, CRTP, PIMPL	31	12	Friends, Nested Classes & Other Tools	33
13	OOP Design: SOLID, Patterns, UML & Anti-Patterns	35	14	Interview Q&A Bank	39
15	Practice Problems & Mini Designs	62	16	Revision: Cheat Sheets, Glossary & Checklist	69

01 Classes, Objects & Encapsulation

Everything in this handbook builds on three words: class, object, encapsulation. Get these precise now and every later chapter gets easier.

1.1 What is OOP, and why does it exist?

Object-Oriented Programming (OOP) is a way of organizing code around **objects** — bundles of data and the functions that operate on that data — instead of organizing it purely around a sequence of steps (procedural programming).

Before OOP, a "procedural" program kept its data in plain structures and passed that data around to free-standing functions. Nothing stopped any function, anywhere in the program, from reaching into that data and changing it incorrectly. As programs grew, this became a maintenance nightmare: to understand whether a piece of data was ever in an invalid state, you had to read *every function in the program* that touched it.

OOP's core idea: put the data and the operations that are allowed to touch it in the same place (a **class**), and make the rest of the program go through a controlled interface. This is what makes large codebases maintainable.

WHY IT EXISTS

OOP is a response to a scaling problem: as codebases grow, uncontrolled access to shared data becomes the #1 source of bugs. Bundling data with its rules, and hiding the data itself, is how OOP solves that.

1.2 Classes and objects

A **class** is a blueprint — a type you define that describes what data (**state**) and what operations (**behavior**) something will have. An **object** is one concrete instance of a class, living somewhere in memory, with its own copy of the state.

```
class BankAccount {
public:
    explicit BankAccount(double balance) : balance_(balance) {}
    void deposit(double amount) { balance_ += amount; }
    double balance() const { return balance_; }
private:
    double balance_; // state
};

BankAccount alice(100.0); // alice is an OBJECT of class BankAccount
BankAccount bob(50.0);   // a completely separate object
alice.deposit(25.0);      // alice.balance_ becomes 125.0; bob is unaffected
```

The keyword `class` declares the blueprint. Each of `alice` and `bob` is a distinct object: same layout, independent state.

INTERNAL MECHANICS

At compile time, a class definition produces no memory by itself — it's a type description the compiler uses to lay out objects and resolve member calls. Only when you write `BankAccount alice(100.0);` does memory actually get reserved (on the stack here) and the constructor run to fill it in. `sizeof(BankAccount)` is a compile-time constant; the number of *objects* is a runtime fact the compiler knows nothing about.

1.3 `struct` vs. `class`

In C++, `struct` and `class` are **the same construct** with one different default:

KEYWORD	DEFAULT MEMBER ACCESS	DEFAULT INHERITANCE
<code>struct</code>	public	public
<code>class</code>	private	private

Everything else — methods, constructors, virtual functions, templates — works identically on both. Convention: use `struct` for passive data bundles with no invariants (e.g. a `Point{x,y}`), and `class` when the type enforces rules about its own data.

1.4 Access specifiers

SPECIFIER	VISIBLE TO
<code>public</code>	Anyone holding an object of this type
<code>protected</code>	This class and any class that inherits from it
<code>private</code>	Only this class's own member functions (and its <code>friend</code> s)

1.5 Encapsulation and invariants

Encapsulation is the bundling of data with the code that operates on it, combined with restricting direct outside access to that data. The real payoff isn't "hiding" for its own sake — it's protecting an **invariant**: a condition that must always hold true for an object.

In the `BankAccount` example, if `balance_` were public, nothing would stop code anywhere from writing `alice.balance_ = -500;`. Because it's private and reachable only through `deposit()`, the class itself becomes the one place that invariant ("balance never goes negative through a bad write") can be enforced — assuming `deposit()` / `withdraw()` are actually written to check it.

```
class BankAccount {
public:
    explicit BankAccount(double balance) : balance_(balance) {}
    bool withdraw(double amount) {
        if (amount > balance_) return false; // invariant: balance_ never goes negative
        balance_ -= amount;
        return true;
    }
private:
    double balance_;
};
```

COMMON MISTAKE

Writing a public getter *and* setter for every private field ("getter/setter soup") isn't encapsulation — it just re-exposes the field through two functions instead of one, with none of the invariant protection. Only add a setter if there's a real rule it enforces; otherwise ask whether the field should just be public (if it's a passive struct) or whether the operation belongs as a named method (`deposit()` , not `setBalance()`).

BEST PRACTICES

Default to `private` data. Expose behavior (named methods that express what the object *does*), not raw field access. Keep a class's invariant checkable by reading only its own member functions — if you need to read code elsewhere in the program to know whether an object is in a valid state, encapsulation has already broken down.

MINI QUIZ — CHECK YOUR UNDERSTANDING

1. What's the one default-access difference between `struct` and `class` ?
2. Why does exposing a public setter for every private field defeat the purpose of encapsulation?
3. True or false: a class definition by itself allocates memory. Explain.
4. Rewrite `BankAccount` so `deposit()` rejects negative amounts. What invariant are you now protecting?

02 Constructors, Destructors & Object Lifecycle

Every object is born (constructed), lives, and dies (deconstructed). C++ gives you precise, deterministic control over both ends of that lifetime — this is one of the biggest differences from garbage-collected OOP languages.

2.1 Constructors

A **constructor** is a special member function that runs automatically when an object is created. Its job is to establish the object's invariants before any other code can touch it. A constructor has the same name as the class and no return type.

CONSTRUCTOR	ANSWERS	SIGNATURE SHAPE
Default	"Make one with no arguments"	<code>ClassName()</code>
Parameterized	"Make one from these values"	<code>ClassName(int x, ...)</code>
Copy	"Make one that's a copy of that one"	<code>ClassName(const ClassName&)</code>
Move	"Make one by cannibalizing that temporary"	<code>ClassName(ClassName&&)</code>
Delegating	"Build myself by calling another of my own constructors first"	<code>ClassName() : ClassName(0) {}</code>

Copy and move get a full chapter later (Ch.8 — Rule of 0/3/5). For now, focus on default/parameterized construction and **member initializer lists**:

```
class Point {
public:
    Point(double x, double y) : x_(x), y_(y) {} // initializer list
private:
    double x_, y_;
};
```

The initializer-list form (`: x_(x), y_(y)`) directly *constructs* each member from the given value. Writing `x_ = x;` in the constructor body instead first default-constructs `x_`, then assigns — wasted work for class-typed members, and outright required (not optional) for two cases: a `const` member and a reference member, neither of which can be assigned to after construction.

COMMON MISTAKE

Members initialize in **declaration order** inside the class, never in the order they're listed in the initializer list. Listing them out of order doesn't reorder execution — it just confuses readers (and many compilers warn about it with `-Wreorder`).

2.2 Delegating constructors (C++11)

```
class Rectangle {
public:
    Rectangle() : Rectangle(1.0, 1.0) {}           // delegates to the constructor below
    Rectangle(double w, double h) : w_(w), h_(h) {}
private:
    double w_, h_;
};
```

Delegating constructors avoid duplicating initialization logic across multiple constructors of the same class.

2.3 Destructors

A **destructor** (`~ClassName()`) runs automatically at the end of an object's lifetime — when a stack object goes out of scope, or when `delete` is called on a heap object. Its job is to release whatever the object acquired.

```
class FileHandle {
public:
    explicit FileHandle(const std::string& path) {
        file_ = std::fopen(path.c_str(), "r");    // constructor: acquire
    }
    ~FileHandle() {
        if (file_) std::fclose(file_);           // destructor: release, always
    }
private:
    FILE* file_;
};
```

INTERNAL MECHANICS – THE OBJECT LIFECYCLE

For a stack object: memory is reserved when the enclosing scope is entered, the constructor runs at the point of definition, the object is usable, and the destructor runs automatically — in exact reverse order of construction — when control leaves the scope (normal return *or* an exception propagating through). For a heap object created with `new`, the constructor runs immediately after allocation, but the destructor only runs when you explicitly call `delete` (or a smart pointer does it for you — Ch.8). Forgetting to `delete` a raw `new` never runs the destructor at all — this is what a memory leak actually is.

```
Stack object lifecycle:
{
    FileHandle f("log.txt");    // <- constructor runs HERE
    f.write(...);
}                                // <- destructor runs HERE, automatically,
                                // even if an exception was thrown above
```

2.4 Construction and destruction order (inheritance preview)

When a derived object is built: base subobject constructors run first, then members (in declaration order), then the derived constructor's own body. Destruction is the *exact reverse*: derived's own body, then members, then base. This matters enough that it comes back in Ch.5 and repeatedly in the interview bank.

BEST PRACTICES

Prefer member initializer lists over body assignment — always for `const` /reference members, and for style everywhere else. Keep constructors focused on establishing invariants, not doing heavy work (a constructor that can fail meaningfully should usually throw, not leave the object half-built). Make single-argument constructors `explicit` unless you specifically want implicit conversions (Ch.3 covers why).

MINI QUIZ

1. Why must a `const` member be initialized in the initializer list, never the constructor body?
2. In what order do members initialize if the initializer list lists them out of declaration order?
3. If a stack object's constructor throws partway through, does its destructor run? What if the constructor completes successfully but a later statement in the same scope throws?
4. What problem do delegating constructors solve?

03 The `this` Pointer, Static Members & Const-Correctness

Three mechanical building blocks that show up in almost every class you'll write from here on.

3.1 The `this` pointer

Every non-static member function implicitly receives a pointer to the object it was called on, named `this`. Its type is `ClassName*` (or `const ClassName*` inside a `const` method).

```
class Builder {
public:
    Builder& setWidth(int w) { width_ = w; return *this; } // enables chaining
    Builder& setHeight(int h) { height_ = h; return *this; }
private:
    int width_ = 0, height_ = 0;
};
Builder b;
b.setWidth(10).setHeight(20); // chained because each call returns *this
```

`this` is mostly implicit — you rarely write `this->member` unless disambiguating from a same-named parameter, or (as above) returning `*this` to enable method chaining.

INTERNAL MECHANICS

`this` is not magic — under the hood, the compiler passes the object's address as a hidden first argument to every non-static member function call. `obj.method(x)` compiles roughly to `ClassName::method(&obj, x)`. This is exactly why static member functions (next section) don't have a `this`: they aren't called on any particular object, so there's no address to pass.

3.2 Static data members

A `static` data member belongs to the *class itself*, not to any one object — exactly one copy exists, shared by every instance.

```
class Widget {
public:
    Widget() { ++count_; }
    ~Widget() { --count_; }
    static int count() { return count_; }
private:
    static inline int count_ = 0; // C++17: define + initialize in-class
};
// Widget::count() reflects how many Widgets currently exist, across ALL objects
```

Before C++17, a `static` member's definition (with its initializer) had to live in exactly one `.cpp` file; `inline` (C++17) lets it be defined directly in the header without violating the one-definition rule.

3.3 Static member functions

A `static` member function has **no** `this` — it can only touch other static members, and is called via `ClassName::function()` without needing an object at all.

3.4 Const-correctness

A `const` member function promises not to modify the object it's called on. The compiler enforces this promise at compile time.

```
class Point {
public:
    double x() const { return x_; }    // promises: does NOT modify *this
    void setX(double x) { x_ = x; }    // non-const: may modify
private:
    double x_;
};

void printPoint(const Point& p) {    // p cannot be modified, no copy made
    std::cout << p.x();            // requires x() to be const -- it is
}
```

A `const` object (or a `const&`) can only call `const` member functions. This is what makes `const&` parameters actually safe: the compiler, not just convention, guarantees the function can't mutate what you passed it.

3.5 mutable

`mutable` marks a member as modifiable even from inside a `const` method — reserved for state that's an internal implementation detail (a cache, a lock) rather than part of the object's logical, externally visible value.

```
class ExpensiveComputation {
public:
    int result() const {
        if (!cached_) { cached_value_ = computeExpensive(); cached_ = true; }
        return cached_value_;
    }
private:
    mutable bool cached_ = false;
    mutable int cached_value_ = 0;
};
```

COMMON MISTAKE

Using `mutable` to sneak around const-correctness for members that *are* part of an object's logical state (e.g. making a whole struct's fields `mutable` so a const method can "just edit them") is an abuse of the keyword — reserve it narrowly for caches and similar bookkeeping that a caller can't observe from the outside.

BEST PRACTICES

Mark every member function `const` that doesn't need to modify the object — it costs nothing and documents intent, and it's what lets callers pass you `const&` safely. Prefer `const&` parameters for anything larger than a couple of machine words unless you specifically want a copy.

MINI QUIZ

1. What hidden argument does the compiler pass into every non-static member function call?
2. Why can't a static member function access `this` ?
3. Why does marking an accessor `const` matter for callers who only have a `const&` to your object?
4. Give one legitimate use of `mutable` and one abuse of it.

04 Abstraction & Interfaces

The third pillar, and the one that sets up everything in Chapters 5–6.

4.1 What is abstraction?

Abstraction means exposing only what a caller needs to know, and hiding how it's actually done. A driver uses a steering wheel and pedals without knowing how the engine's fuel injection works — that's abstraction. In C++, the primary tool for abstraction is the **abstract class**: a class with at least one **pure virtual function**.

4.2 Pure virtual functions and abstract classes

```
class Shape {    // an abstract class -- an interface
public:
    virtual double area() const = 0;    // pure virtual -- no body required
    virtual ~Shape() = default;
};
// Shape s;    -- COMPILE ERROR: cannot instantiate an abstract class
```

The `= 0` marks `area()` as pure virtual: `Shape` declares that every shape must provide an area, without saying how. A class with any pure virtual function is **abstract** and cannot be instantiated directly. A class that inherits from `Shape` must override `area()` to become concrete (instantiable) — or remain abstract itself.

```
class Circle : public Shape {
public:
    explicit Circle(double r) : radius_(r) {}
    double area() const override { return 3.14159 * radius_ * radius_; }
private:
    double radius_;
};
Circle c(2.0);    // OK -- Circle is concrete, it implements area()
```

INTERNAL MECHANICS (PREVIEW OF CH.7)

C++ has no dedicated "interface" keyword. What other languages call an interface, C++ expresses as a class with only pure virtual functions and no data — the idiom, not a language feature. Mechanically, a pure virtual function still gets a vtable slot (Ch.7); marking it `= 0` just tells the compiler "there is no default implementation, and this class can't be instantiated until something fills the slot."

4.3 Can a pure virtual function have a body?

Yes — surprisingly. `virtual void f() = 0;` can still be defined elsewhere and called explicitly as `Base::f()`. This matters most for destructors: a pure virtual destructor (used to force a class to be abstract when it otherwise has no other pure virtual function) still needs a body, because every destructor in an inheritance chain unconditionally runs during destruction.

4.4 Interfaces vs. abstract classes with partial implementation

A class can mix pure virtual functions (the contract) with regular virtual or non-virtual functions (shared implementation) and data members. This is still "abstract" in C++'s sense, but it's doing

double duty: part interface, part reusable base behavior. Chapter 5 discusses when that mixing is appropriate and when it isn't.

CONCEPT	HAS DATA?	HAS IMPLEMENTATION?	INSTANTIABLE?
Pure interface (idiom)	No	No	No
Abstract class (general)	Maybe	Maybe (partial)	No
Concrete class	Yes	Yes (complete)	Yes

COMMON MISTAKE

Forgetting a virtual destructor on an abstract base that will be deleted through a base pointer. This is a bigger deal than it looks — covered fully with a traced example in Ch.6.5, because it's one of the single most common interview gotchas in all of C++ OOP.

BEST PRACTICES

Keep interfaces small and focused (this is the "I" in SOLID — Ch.13). Prefer pure interfaces (no data, no implementation) when you specifically want polymorphic substitution with no shared state; reach for a partially-implemented abstract base only when there's genuine common behavior worth sharing.

MINI QUIZ

1. What makes a class "abstract" in C++'s specific sense?
2. Can you call `Base::f()` explicitly even if `f` is pure virtual? Why would you want to?
3. Does C++ have a dedicated `interface` keyword? What's the idiom instead?

05 Inheritance Fundamentals

The fourth pillar. Inheritance is powerful and overused — this chapter builds the mechanics; Ch.9 covers when *not* to reach for it.

5.1 What is inheritance?

Inheritance lets a **derived class** acquire the members of a **base class**, modeling an "is-a" relationship: a `Dog` is an `Animal`.

```
class Animal {
public:
    void breathe() { std::cout << "breathing\n"; }
protected:
    int age_ = 0;
};
class Dog : public Animal {    // Dog IS-A Animal
public:
    void bark() { std::cout << "woof\n"; }
};
Dog d;
d.breathe();    // inherited from Animal
d.bark();       // Dog's own
```

5.2 The three inheritance access levels

C++ lets you choose how a base class's access levels carry forward into the derived class:

INHERITANCE	PUBLIC MEMBERS OF BASE BECOME	PROTECTED MEMBERS BECOME
<code>public</code>	public	protected
<code>protected</code>	protected	protected
<code>private</code>	private	private

```
class Dog : public Animal {};    // public members stay public (the normal case)
class Cat : protected Animal {}; // public members become protected
class Fish : private Animal {};  // public members become private
```

`public` inheritance is the overwhelming default — it's the only one that preserves genuine "is-a" substitutability (Ch.6). `protected` / `private` inheritance are rare in practice; when people reach for them they usually actually want composition (Ch.9).

5.3 Overriding a base method

```
class Animal {
public:
    virtual void speak() const { std::cout << "...\\n"; }
};
class Dog : public Animal {
public:
    void speak() const override { std::cout << "Woof!\\n"; }
};
```

This is a preview — Chapter 6 is entirely about what `virtual` actually does and why it's needed for this to work polymorphically.

INTERNAL MECHANICS

A derived object's memory begins with its base subobject, followed by its own members (Ch.7 draws this out precisely). This is why a `Dog*` converts implicitly to an `Animal*` with no adjustment needed in the common single-inheritance case — the `Animal` part genuinely starts at the same address as the whole `Dog` object.

5.4 Hiding vs. overriding

If a derived class declares a non-virtual function with the same name as a base function, it **hides** the base version rather than overriding it — calls resolve based on the static (declared) type, not dynamically. This is a frequent source of confusion; Ch.6 covers static vs. dynamic binding precisely.

COMMON MISTAKE

Inheriting just to reuse a few methods, with no genuine is-a relationship, is a warning sign a interviewer will probe. A `Stack` is not a `std::vector` just because it can reuse its storage internally — it *has* one (composition, Ch.9).

BEST PRACTICES

Default to `public` inheritance. Ask "is this genuinely an is-a relationship where I need polymorphic substitution?" before inheriting — if the answer is "I just want to reuse some code," reach for composition instead.

MINI QUIZ

1. Which inheritance access level is the default, sensible choice almost all the time?
2. If `Cat` inherits `protected` ly from `Animal` , can code outside `Cat` call an originally-public `Animal` method on a `Cat` object?
3. What's the difference between "hiding" and "overriding" a base method?

06 Polymorphism & Virtual Functions

The pillar interviewers spend the most time on. This chapter gets you to "correct and confident"; Chapter 7 goes underneath it to "I know exactly how this works."

6.1 What is polymorphism?

Polymorphism ("many forms") is the ability to treat objects of different derived types through a common base interface, with the correct derived behavior happening automatically. C++ has two distinct kinds:

	COMPILE-TIME (STATIC)	RUNTIME (DYNAMIC)
Mechanism	Function overloading, templates	<code>virtual</code> functions
Resolved	By the compiler, before the program runs	At runtime, based on the object's actual type
Cost	Zero runtime overhead	One indirect call per dispatch

When people say "polymorphism" unqualified in an OOP conversation, they almost always mean **runtime polymorphism** — the rest of this chapter is about that.

6.2 Virtual functions

```

class Shape {
public:
    virtual double area() const { return 0.0; }    // virtual: overridable, dispatched at runtime
    virtual ~Shape() = default;
};
class Circle : public Shape {
public:
    explicit Circle(double r) : radius_(r) {}
    double area() const override { return 3.14159 * radius_ * radius_; }
private:
    double radius_;
};

Shape* s = new Circle(2.0);
s->area();    // calls Circle::area() at RUNTIME, even though s's static type is Shape*

```

The magic: even though `s`'s declared (static) type is `Shape*`, the call resolves to `Circle::area()` because that's the object's *actual* (dynamic) type. This is runtime polymorphism.

6.3 Overriding vs. overloading — don't confuse these

	OVERRIDING	OVERLOADING
What	Derived class redefines a base <code>virtual</code> function	Multiple functions, same name, different parameters
Resolved	Runtime (dynamic dispatch)	Compile time
Requires <code>virtual</code> ?	Yes	No — unrelated concept

```

// OVERLOADING (compile-time, unrelated to virtual/inheritance):
void print(int x);
void print(const std::string& x);

```

6.4 `override` and `final` (C++11)

```

class Circle : public Shape {
public:
    double area() const override { return 3.14; }
    // double Area() const override; -- COMPILE ERROR: no such virtual in Shape
};
class UnitCircle final : public Circle {};    // no class may inherit from UnitCircle

```

`override` tells the compiler "verify this actually overrides a base virtual function." Without it, misspelling a signature (wrong name, wrong `const`-ness, wrong parameter types) silently creates an unrelated new function instead of overriding anything — and the bug is invisible until you

notice the wrong version running. `final` on a class forbids further inheritance; on a method it forbids further overriding.

6.5 Virtual destructors — the classic gotcha

```
class Base {
public:
    ~Base() { std::cout << "~Base\n"; }           // NOT virtual -- the bug
};
class Derived : public Base {
public:
    ~Derived() { delete[] buffer_; std::cout << "~Derived\n"; }
private:
    int* buffer_ = new int[100];
};
Base* b = new Derived();
delete b;
// Output: "~Base" ONLY -- ~Derived() never runs. buffer_ leaks. Undefined behavior, technically.
```

Without `virtual`, destructor selection is resolved **statically** from the pointer's declared type — exactly like any other non-virtual call. The fix is one word:

```
class Base {
public:
    virtual ~Base() { std::cout << "~Base\n"; }    // fixed
};
// Now: delete b; prints "~Derived\n~Base\n" -- both run, correct order
```

THE RULE

If a class has *any* virtual function, or is ever deleted through a base pointer, give it a virtual destructor. This single rule prevents one of the most common real-world C++ bugs and one of the most common interview trick questions.

INTERNAL MECHANICS (FULL DETAIL IN CH.7)

A non-virtual destructor call is resolved the exact same way any non-virtual function call is: baked into the machine code at compile time, based on the pointer's declared type. A virtual destructor is resolved through the vtable, just like any other virtual call — Chapter 7 shows precisely what that mechanism is.

BEST PRACTICES

Give any class designed to be a polymorphic base (i.e., meant to be inherited from and used through base pointers/references) a `virtual` destructor — even if it's just `virtual ~Base() = default;`. Mark overrides `override` always, with no exceptions.

MINI QUIZ

1. What's the practical difference between overloading and overriding, in one sentence each?
2. Trace exactly why the non-virtual-destructor example above leaks `buffer_`.
3. What compile-time safety does `override` buy you that omitting it doesn't?
4. When should a class's destructor be `virtual`?

07 Internals: vtable, Binding, Memory Layout, Slicing & RTTI

This is the chapter most OOP material skips. Understanding what actually happens in memory is what separates "I can use virtual functions" from "I understand C++ OOP."

7.1 Object memory layout

```
class Point { int x_; int y_; }; // sizeof(Point) == 8

Point p: [ x_ | y_ ]
         offset 0 offset 4

class Animal { int age_; };
class Dog : public Animal { int breed_id_; };

Dog d: [ Animal part (age_) | Dog's own (breed_id_) ]
       offset 0 offset 4
// A Dog* converts implicitly to Animal* -- the Animal subobject
// genuinely starts at the SAME address as the whole Dog object.
```

A derived object's memory is laid out as its base subobject(s) first, followed by its own new members (declaration order). This is why base-pointer conversion is "free" — no arithmetic needed, just reinterpreting the same address with a different type, in the common single-inheritance case.

7.2 The vtable and vptr — how dynamic dispatch actually works

The moment a class has any **virtual** function, the compiler adds one hidden pointer to every object of that type: the **vp**tr (virtual pointer). It also generates, once per class (not per object), a **vtable** — an array of function pointers, one slot per virtual function.

```
Shape's vtable (ONE per class):      Circle's vtable:
[ slot 0: &Shape::area   ]          [ slot 0: &Circle::area   ] <- overridden
[ slot 1: &Shape::~Shape ]          [ slot 1: &Circle::~Circle ]

EVERY OBJECT gets its own vp
```

tr, set by its constructor:
Circle c: [vp

tr | radius_] ---> points at Circle's vtable

A call like shape_ptr->area() compiles to roughly:

1. follow shape_ptr's vp
2. index the FIXED slot number reserved for area() (slot 0 here)
3. call whatever function pointer is stored there

The vp

tr belongs to the *object*, not to the pointer or reference used to access it — so it always resolves to the object's real, dynamic class, regardless of what static type you're viewing it through. That's the entire mechanism behind runtime polymorphism.

WHY THIS DESIGN

This is why a virtual call costs slightly more than a direct call: two memory reads (follow vp

tr, then index the vtable) plus an indirect call through a function pointer, versus a direct compile-time-known jump for a non-virtual call. In hot loops this is occasionally measurable, which is why performance-critical inner loops sometimes deliberately avoid virtual dispatch (see CRTP, Ch.11).

7.3 Static binding vs. dynamic binding

Static binding: the compiler decides which function to call at compile time, purely from the declared (static) type of the expression. **Dynamic binding:** deferred to runtime, resolved via the vtable based on the object's actual type. Non-virtual calls, overloaded calls, and hidden (not overridden) methods are all statically bound; virtual calls made through a pointer or reference are dynamically bound.

7.4 Object slicing

```
void printArea(Shape s) {           // BUG: takes Shape BY VALUE
    std::cout << s.area();
}
Circle c(5.0);
printArea(c); // c is SLICED down to just its Shape part.
// s.area() calls Shape::area(), NOT Circle::area() -- radius_ and the
// vtable identity simply aren't copied; the base copy ctor knows nothing about them.
```

Passing a polymorphic object by value invokes the *base* class's copy constructor, which only knows about the base's own members. This compiles cleanly and runs without crashing — it's just silently wrong.

HOW TO AVOID IT

Pass and return polymorphic types by reference or pointer, never by value. Storing `Circle` objects directly in a `std::vector<Shape>` slices on every single insertion — use `std::vector<std::unique_ptr<Shape>>` instead (Ch.8 covers why `unique_ptr` specifically).

7.5 RTTI: `dynamic_cast` and `typeid`

```
Shape* s = getRandomShape();
if (Circle* c = dynamic_cast<Circle*>(s)) {
    // only enters here if s genuinely points at a Circle (or a class derived from it)
}
if (typeid(*s) == typeid(Circle)) { /* exact-type check */ }
```

RTTI (Runtime Type Information) lets code query an object's actual type at runtime.

`dynamic_cast` is a *safe* downcast — it returns `nullptr` (for pointers) or throws `std::bad_cast` (for references) on failure, and requires the base class to be polymorphic (have at least one virtual function) to work at all. `static_cast` is faster but unsafe — a wrong downcast compiles fine and produces undefined behavior the moment you use it.

WHY HEAVY `DYNAMIC_CAST` USE IS A DESIGN SMELL

Needing `dynamic_cast` frequently, scattered through client code that consumes a base pointer, usually means the base interface is missing a virtual function — add the behavior to the interface instead of branching on concrete type from outside the hierarchy. Occasional, deliberate use (e.g. a visitor-pattern fallback) is fine.

BEST PRACTICES

Never store polymorphic objects by value in a container. Prefer designing the virtual interface to avoid needing `dynamic_cast` at all. If you must query type, prefer `dynamic_cast` over `static_cast` + manual bookkeeping — correctness beats the small performance cost almost always.

MINI QUIZ

1. How many vtables exist for a class with 100 instances? How many vptrs?
2. Walk through, step by step, what `shape_ptr->area()` compiles to when `area()` is virtual.
3. Why does `void printArea(Shape s)` silently call the wrong `area()` for a `Circle` argument?
4. What does `dynamic_cast` require of the base class to work at all?

08 Resource Management: Rule of 0/3/5, RAII, Smart Pointers

C++ has no garbage collector. This chapter is how C++ objects manage memory, files, locks, and sockets safely anyway — arguably the most distinctively "C++" part of C++ OOP.

8.1 The special member functions

Every class has up to five compiler-generatable special member functions: destructor, copy constructor, copy assignment, move constructor, move assignment. The compiler generates default versions of these for you — but those defaults are only correct for certain classes.

RULE	STATEMENT
Rule of 0	Write <i>none</i> of the five — let members (<code>string</code> , <code>vector</code> , smart pointers) manage their own resources. The default, and the goal.
Rule of 3	If you write any one of destructor / copy constructor / copy assignment, you almost certainly need all three (classic pre-C++11 idiom, for classes owning a raw resource).
Rule of 5	C++11 adds move constructor and move assignment to that set. If you need any one of the five, seriously consider all five.

8.2 Deep copy vs. shallow copy

The compiler-generated copy constructor is always a **shallow copy**: it copies each member's value as-is. For a raw pointer member, that copies the *pointer*, not what it points to — both objects then point at the same heap block.

```
class Buffer {
public:
    explicit Buffer(size_t n) : size_(n), data_(new int[n]) {}
    // Compiler-generated copy ctor (if we didn't write our own) would do:
    // data_ = other.data_; -- SHALLOW: both objects now share one buffer
    ~Buffer() { delete[] data_; }
private:
    size_t size_;
    int* data_;
};
```

THE CLASSIC DOUBLE-FREE

Two objects sharing one pointer via a shallow copy: whichever is destroyed first frees the memory correctly. The *second* object's destructor then frees the **same address again** — undefined behavior, often a crash, sometimes silent corruption. The fix is a real, **deep**-copying copy constructor that allocates its own new block:

```
Buffer(const Buffer& other) : size_(other.size_), data_(new int[other.size_]) {
    std::copy(other.data_, other.data_ + size_, data_); // DEEP: own storage
}
```

8.3 Move semantics

Copying duplicates a resource. **Moving** steals it from a source that's about to be destroyed anyway (a temporary, or something explicitly marked with `std::move`) — dramatically cheaper for anything heap-owning.

```
class Buffer {
public:
    explicit Buffer(size_t n) : size_(n), data_(new int[n]) {}
    Buffer(const Buffer& other) : size_(other.size_), data_(new int[other.size_]) {
        std::copy(other.data_, other.data_ + size_, data_);
    }
    Buffer(Buffer&& other) noexcept : size_(other.size_), data_(other.data_) {
        other.size_ = 0;
        other.data_ = nullptr; // source left in a valid, destructible (empty) state
    }
    ~Buffer() { delete[] data_; }
private:
    size_t size_;
    int* data_;
};
```

`Buffer&&` is an **rvalue reference** — it binds to temporaries and to anything explicitly cast with `std::move` (which is just a cast; it moves nothing by itself). Mark move operations `noexcept`: containers like `std::vector` only move elements during reallocation if the move constructor is guaranteed not to throw — otherwise they silently fall back to copying, quietly losing the performance benefit.

8.4 RAI — Resource Acquisition Is Initialization

RAII ties a resource's lifetime to an object's lifetime: acquire the resource in the constructor, release it in the destructor.

```
class LockGuard {
public:
    explicit LockGuard(std::mutex& m) : mutex_(m) { mutex_.lock(); }
    ~LockGuard() { mutex_.unlock(); } // runs even if an exception is thrown in between
private:
    std::mutex& mutex_;
};

void critical() {
    LockGuard guard(myMutex);
    doWork(); // if this throws, guard's destructor STILL unlocks the mutex
}
```

Because C++ guarantees local objects' destructors run during stack unwinding (Ch.2), RAII makes resource leaks on the exception path structurally hard to write by accident — the resource is released no matter how the scope is exited.

8.5 Smart pointers

TYPE	OWNERSHIP	USE FOR
<code>unique_ptr<T></code>	Sole owner, non-copyable, movable	Default choice for any owned heap object
<code>shared_ptr<T></code>	Shared, reference-counted	Genuinely shared ownership
<code>weak_ptr<T></code>	Non-owning observer of a <code>shared_ptr</code>	Breaking reference cycles (Ch.14 has a full traced example)

```
auto u = std::make_unique<Widget>(args); // preferred over `new Widget(args)`
auto s = std::make_shared<Widget>(args);
```

WHY PREFER `MAKE_UNIQUE` / `MAKE_SHARED`

They avoid a raw `new` as a separate, nameable expression — which removes a subtle leak window if a constructor argument throws mid-expression before the pointer is safely owned — and `make_shared` additionally allocates the control block and the object in a single heap allocation instead of two.

8.6 Exception safety guarantees

GUARANTEE	MEANING
Basic	On exception, no leaks; object stays in a valid (but unspecified) state
Strong	On exception, the operation has no effect at all — as if it were never called
Nothrow	The operation is guaranteed never to throw

BEST PRACTICES

Prefer Rule of 0 — let `string` / `vector` / smart pointers do the resource management, and write no special member functions at all. If a class genuinely owns a raw resource, write the full Rule of 5, mark move operations `noexcept`, and prefer `make_unique` / `make_shared` over raw `new` everywhere.

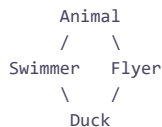
MINI QUIZ

1. Why is the compiler-generated copy constructor dangerous for a class holding a raw owning pointer?
2. What does marking a move constructor `noexcept` actually change about how `std::vector` treats it?
3. What specific guarantee does RAII give you on the exception path that manual acquire/release doesn't?
4. When would you reach for `shared_ptr` instead of `unique_ptr`?

09 Advanced Inheritance & Composition

Where single inheritance gets complicated, and why most experienced C++ engineers reach for composition first.

9.1 Multiple inheritance and the diamond problem



```
class Animal { public: int age_; };
class Swimmer : public Animal {};
class Flyer : public Animal {};
class Duck : public Swimmer, public Flyer {};
// Duck d; d.age_; -- AMBIGUOUS: TWO separate Animal subobjects exist inside d
```

Without intervention, `Duck` contains **two independent `Animal` subobjects** — one arriving via `Swimmer`, one via `Flyer`. `d.age_` is ambiguous, and semantically, changing one copy doesn't change the other — clearly not what "a Duck is one Animal" should mean.

9.2 Virtual inheritance — the fix

```
class Animal { public: int age_; };
class Swimmer : public virtual Animal {};
class Flyer : public virtual Animal {};
class Duck : public Swimmer, public Flyer {};
// Now Duck has exactly ONE Animal subobject, shared by both paths.
```

virtual inheritance makes the base a single shared subobject — resolving the ambiguity, at the cost of extra indirection and more delicate construction-order rules (the *most-derived* class becomes directly responsible for initializing the virtual base, bypassing the intermediate classes).

9.3 Composition vs. inheritance

	INHERITANCE ("IS-A")	COMPOSITION ("HAS-A")
Coupling	Tight — derived depends on base internals	Loose — interacts only through the member's public interface
Flexibility	Fixed at compile time	Can swap the component at runtime
Use when	Genuine is-a relationship, need polymorphic substitution	Reuse without commitment to a rigid hierarchy

```
class Engine { public: void start() {} };
class Car { // Car HAS-A Engine, not IS-A
public:
    void start() { engine_.start(); }
private:
    Engine engine_;
};
```

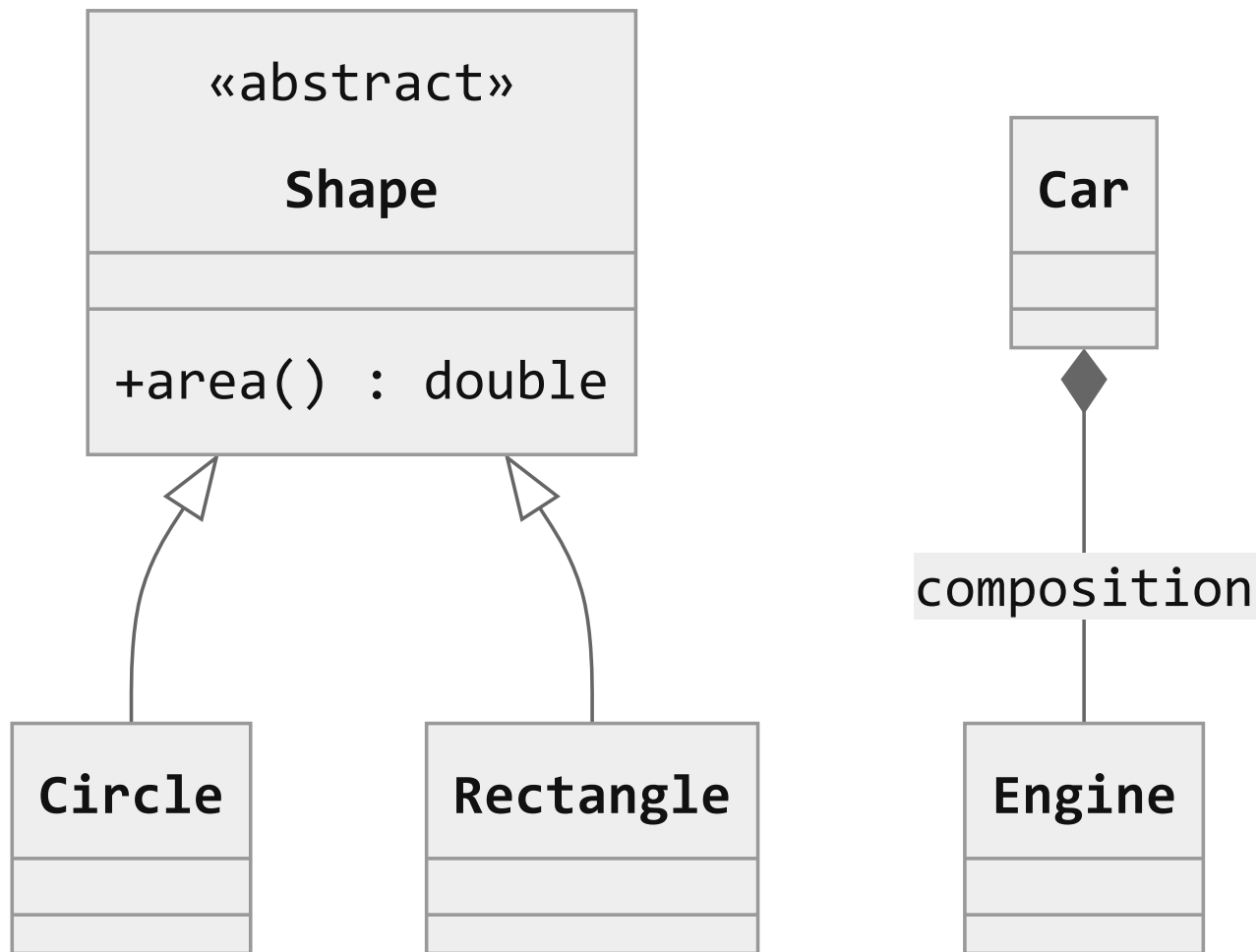
THE DEFAULT GUIDELINE

"Favor composition over inheritance" — reach for inheritance only when you specifically need polymorphic substitution (a base pointer/reference genuinely needs to refer interchangeably to any derived type). If you just want to reuse an implementation, hold it as a member instead.

9.4 Interfaces vs. implementation inheritance

Inheriting a pure-abstract class (all pure virtual, no data — Ch.4) is **interface inheritance**: you inherit a contract, not behavior. Inheriting a class with real implementation is **implementation inheritance**: you inherit behavior too, and any bug or later change in the base ripples silently into

every derived class. Prefer interface inheritance for polymorphism; reserve implementation inheritance for genuinely stable hierarchies.



COMMON MISTAKE

Reaching for multiple inheritance to combine two unrelated "capabilities" (e.g. `class Logger : public FileWriter, public NetworkClient`) when neither is genuinely an is-a relationship. Prefer composition, or, if polymorphism is truly needed, small single-purpose interfaces.

BEST PRACTICES

Default to composition. Reach for single inheritance when substitution is genuinely needed. Reach for multiple inheritance only for combining multiple *pure interfaces* (no shared data, no diamond risk) — that specific case is safe and common (e.g. inheriting from several ABCs).

MINI QUIZ

1. Why does non-virtual multiple inheritance from a common base create ambiguity?
2. What does `virtual` inheritance actually change about the object's memory layout?
3. Give one example each of a genuine is-a and a genuine has-a relationship from a domain you know.

10 Operator Overloading

How to make your own types feel like built-in ones — and the discipline required not to make them confusing.

10.1 What to overload, and what not to

GOOD CANDIDATES	POOR CANDIDATES
<code>+</code> on a <code>Vector2D</code> (matches intuition)	<code>+</code> meaning "merge" on unrelated types (surprising)
<code>==</code> for value comparison	<code>&&</code> / <code> </code> with custom short-circuit semantics (loses real short-circuiting)
<code><<</code> for printing	Overloading everything "because you can"

Follow the **principle of least astonishment**: an overloaded operator should behave the way a reader's intuition expects from the built-in version.

10.2 `operator=` and the self-assignment guard

```
Buffer& operator=(const Buffer& other) {  
    if (this == &other) return *this;    // guard: a = a must be a no-op, not a self-destruct  
    delete[] data_;  
    size_ = other.size_;  
    data_ = new int[size_];  
    std::copy(other.data_, other.data_ + size_, data_);  
    return *this;  
}
```

Without the guard, `a = a` would `delete[]` the very data it's about to read from — a classic use-after-free.

10.3 `operator==` and comparisons

```
class Point {
public:
    bool operator==(const Point& o) const { return x_ == o.x_ && y_ == o.y_; }
    bool operator!=(const Point& o) const { return !(*this == o); }
private:
    int x_, y_;
};
```

(C++20 note: `operator<=>`, the "spaceship" operator, can generate all six relational operators from one function via `= default`. Not available in C++17 – listed here for awareness.)

10.4 Stream operators must be free functions

```
class Point {
    friend std::ostream& operator<<(std::ostream& os, const Point& p);
    // ...
};
std::ostream& operator<<(std::ostream& os, const Point& p) {
    return os << "(" << p.x_ << ", " << p.y_ << ")";
}
```

COMMON MISTAKE

`operator<<` cannot be a member function — the left operand of `os << p` would then have to be a `Point`, not an `ostream`. It must be a free function (often a `friend` for private-member access — Ch.12).

10.5 The function-call operator (functors)

```
class IsEven {
public:
    bool operator()(int x) const { return x % 2 == 0; }
};
IsEven isEven;
isEven(4);    // true -- object called like a function ("functor")
std::count_if(v.begin(), v.end(), IsEven{});    // works anywhere a callable is expected
```

BEST PRACTICES

Overload only when it genuinely matches reader intuition. Always guard `operator=` against self-assignment. Keep comparison operators consistent with each other (if `a == b`, then `!(a != b)`).

MINI QUIZ

1. Why must `operator<<` for printing be a free function, not a member?
2. What specifically goes wrong if `operator=` has no self-assignment guard?
3. What is a "functor," and how does it differ from a plain function?

11 Compile-Time OOP: Templates, CRTP, PIMPL & Type Erasure

C++'s answer to "what if I want polymorphism-like behavior without the vtable cost, or want to hide implementation details entirely?"

11.1 Templates and OOP — a quick bridge

A **class template** lets a class be parameterized by type, generating a distinct concrete class per instantiation at compile time — a form of compile-time genericity, distinct from (but often combined with) runtime polymorphism.

```
template<typename T>
class Box {
public:
    explicit Box(T value) : value_(value) {}
    T get() const { return value_; }
private:
    T value_;
};
Box<int> b1(42);
Box<std::string> b2("hi"); // Box<int> and Box<string> are two DIFFERENT, unrelated classes
```

11.2 CRTP — the Curiously Recurring Template Pattern

```

template <typename Derived>
class Shape {
public:
    double area() const { return static_cast<const Derived*>(this)->areaImpl(); }
};
class Circle : public Shape<Circle> {    // Circle inherits from Shape<Circle>
public:                                  // -- "curiously recurring"
    double areaImpl() const { return 3.14159 * r_ * r_; }
private:
    double r_ = 1.0;
};

```

CRTP achieves "polymorphism"-like dispatch resolved entirely at **compile time** via templates — no vtable, no vptr per object, no indirect call. The trade-off: no true runtime polymorphism (you can't hold a single `vector` of mixed `Shape<?>` concrete types the way you can with real virtual dispatch, because each is a distinct instantiated type), and every concrete type produces a separate compiled class.

INTERNAL MECHANICS

`static_cast<const Derived*>(this)` is safe specifically because of the CRTP contract: `Circle` publicly inherits from `Shape<Circle>`, so any `Shape<Circle>` object being called through really is (as its complete object) a `Circle`. The compiler resolves `areaImpl()` at compile time because it knows the concrete type `Derived` from the template parameter — no runtime lookup needed.

11.3 PIMPL — Pointer to Implementation

```

// Widget.h
class Widget {
public:
    Widget();
    ~Widget();           // declared here, DEFINED in Widget.cpp
private:
    class Impl;           // incomplete type -- fine in a header
    std::unique_ptr<Impl> pImpl_;
};

// Widget.cpp
class Widget::Impl { /* real private members/implementation, hidden from the header */ };
Widget::Widget() : pImpl_(std::make_unique<Impl>()) {}
Widget::~~Widget() = default;    // Impl is a COMPLETE type here -- ok to destroy it

```

PIMPL hides a class's private implementation behind an opaque pointer, so changing those internals doesn't force every file that `#include`s the header to recompile — a real, significant build-time win in large codebases.

WHY THE DESTRUCTOR MUST LIVE IN THE .CPP FILE

Destroying a `unique_ptr<Impl>` requires `Impl` to be a *complete* type at the point of destruction. In the header, `Impl` is deliberately only forward-declared — so the destructor (even a defaulted one) must be defined in the `.cpp` file, where `Impl`'s full definition is visible.

11.4 Type erasure (brief)

Type erasure is the technique behind `std::function` and `std::any`: hiding a concrete type behind a uniform interface at runtime, without requiring the stored types to share a common inherited base. It combines templates (to capture the concrete type once, at the call site) with an internal virtual-like dispatch (often a hand-rolled vtable-equivalent) to erase that type afterward. Full implementation is advanced/out of scope here — recognize the term and the concept for interviews.

BEST PRACTICES

Reach for CRTP only when profiling shows virtual dispatch is a real bottleneck, or when you specifically want compile-time policy injection. Use PIMPL for public library headers where build-time isolation matters; it's overkill for small internal classes.

MINI QUIZ

1. Why is `Box<int>` a different class from `Box<std::string>`, unlike a single `Shape` base class covering many derived shapes?
2. What specifically does CRTP trade away to avoid vtable overhead?
3. Why must PIMPL's destructor be defined in the `.cpp` file rather than defaulted in the header?

12 Friends, Nested Classes & Other Tools

Smaller tools that round out the C++ OOP toolbox — each narrow, each easy to misuse if reached for too often.

12.1 **friend** functions and classes

```
class Matrix {
    friend Matrix operator+(const Matrix& a, const Matrix& b);    // can touch private data_
private:
    std::vector<std::vector<double>> data_;
};
Matrix operator+(const Matrix& a, const Matrix& b) {
    Matrix result;
    // ... reads a.data_ and b.data_ directly, despite not being a member
    return result;
}
```

LEGITIMATE USE CASE

A binary operator needing symmetric private access to both operands

A tightly coupled helper or unit-test class

RISK

Breaks encapsulation — use narrowly and deliberately

Overuse signals the class boundary itself is drawn wrong

INTERVIEW NOTES

Friendship is neither inherited (a derived class doesn't inherit its base's friends) nor mutual (A declaring B a friend does not make A a friend of B, even implicitly).

12.2 Nested classes

```
class LinkedList {
public:
    class Iterator {    // nested class -- scoped to LinkedList::Iterator
    public:
        void next();
    };
    Iterator begin();
private:
    struct Node { int value; Node* next; };    // implementation detail, hidden inside LinkedList
};
```

A nested class is a namespacing tool: it groups a helper type that only makes sense in the context of its enclosing class, and keeps it out of the surrounding namespace. A nested class has no special access to the outer class's *instances* just by being nested — it needs its own reference/pointer to one, same as any other class.

12.3 Scoped enums (`enum class`)

```
enum class Color { Red, Green, Blue };    // C++11: scoped, strongly typed
Color c = Color::Red;                     // must qualify -- no implicit int conversion
// int x = Color::Red;  -- COMPILE ERROR (unlike a plain 'enum')
```

Plain `enum` members leak into the enclosing scope and implicitly convert to `int` — a frequent source of accidental bugs when two unrelated enums both have a member named e.g. `None`.

`enum class` fixes both problems and should be the default choice in modern C++.

12.4 Empty Base Optimization (EBO)

An empty class (no non-static data members) still normally has `sizeof >= 1`, so that distinct objects get distinct addresses. But as a *base* subobject specifically, the compiler is permitted to give it zero size, since the derived object already guarantees a unique address on its own. This lets stateless policy/base classes (a custom empty allocator, an empty deleter) be inherited from at no size cost — versus paying for them as a data member, which cannot be optimized away the same way.

BEST PRACTICES

Use `friend` sparingly and only where symmetric access genuinely requires it. Prefer `enum class` over plain `enum` in all new code. Use nested classes for tightly-scoped helpers (iterators, node types), not as a general organizational tool.

MINI QUIZ

1. If class `A` declares class `B` a friend, can `B`'s private members be accessed by `A`?
2. What two problems does `enum class` fix relative to a plain `enum`?
3. Does nesting a class inside another automatically give it access to the outer class's instance data?

13 OOP Design: SOLID, Patterns, UML & Anti-Patterns

The mechanics from Ch.1–12, organized into judgment: how to design an OOP system that stays maintainable as it grows.

13.1 SOLID principles

LETTER	PRINCIPLE	IN ONE SENTENCE
S	Single Responsibility	A class should have one reason to change.
O	Open/Closed	Open for extension, closed for modification — add behavior via new derived classes, not by editing existing ones.
L	Liskov Substitution	A derived object must be usable anywhere a base object is expected, without surprising behavior.
I	Interface Segregation	Prefer several small, focused interfaces over one large one that forces unused methods on implementers.
D	Dependency Inversion	Depend on abstractions (interfaces), not concrete implementations.

```
// Liskov violation: Square "is-a" Rectangle mathematically, but not behaviorally here
class Rectangle {
public:
    virtual void setWidth(double w) { width_ = w; }
    virtual void setHeight(double h) { height_ = h; }
protected:
    double width_ = 0, height_ = 0;
};
class Square : public Rectangle {
public:
    void setWidth(double w) override { width_ = height_ = w; } // surprising side effect!
    void setHeight(double h) override { width_ = height_ = h; }
};
// Code that does rect->setWidth(5); rect->setHeight(4); expecting area==20
// silently gets area==16 if rect actually points at a Square. LSP violated.
```

INTERVIEW NOTES

The Square/Rectangle example is the single most common Liskov Substitution question. The lesson isn't "inheritance is bad" — it's that is-a in the mathematical sense doesn't guarantee is-a in the *behavioral* sense the base class's contract promises.

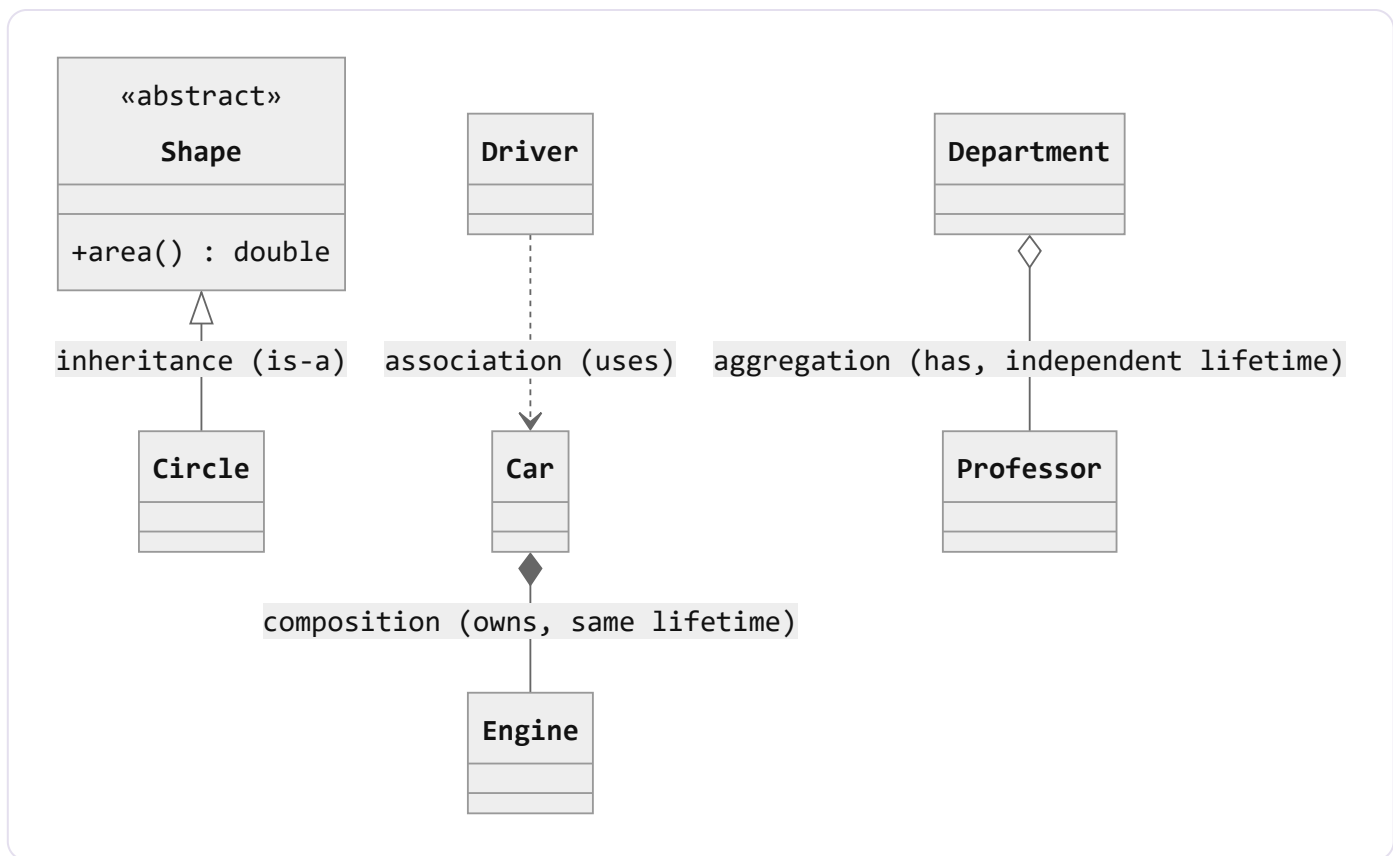
13.2 A quick tour of the most-asked design patterns

PATTERN	PROBLEM IT SOLVES	OOP MECHANISM USED
Factory	Decouple object creation from usage	Virtual function returning a base pointer
Strategy	Swap an algorithm at runtime	Composition + a small interface
Observer	Notify many objects when one changes	A list of base-pointer callbacks/interfaces
Singleton	Exactly one instance, globally reachable	Static local variable (Meyers' Singleton) + private constructor
Decorator	Add behavior to an object without subclassing	Composition: wrap one implementation of an interface in another

```
// Meyers' Singleton -- thread-safe in C++11+ (static local init is guaranteed once)
class Logger {
public:
    static Logger& instance() {
        static Logger single; // constructed on first call, exactly once, thread-safe since C++11
        return single;
    }
private:
    Logger() = default;
    Logger(const Logger&) = delete; // prevent copies
    Logger& operator=(const Logger&) = delete;
};
```

(A full pattern catalog with GoF classification belongs in a dedicated LLD/design-patterns resource — this section covers just the OOP mechanism each pattern rests on, which is what a C++ OOP interview usually probes.)

13.3 Reading a UML class diagram



RELATIONSHIP	ARROW	LIFETIME COUPLING
Inheritance	Hollow triangle	N/A (type relationship)
Composition	Filled diamond	Owned dies with owner
Aggregation	Hollow diamond	Independent
Association	Plain/dashed arrow	Independent, no ownership

13.4 Common OOP anti-patterns

ANTI-PATTERN	WHAT IT LOOKS LIKE	FIX
God class	One class doing everything, hundreds of methods	Split by responsibility (SRP)
Getter/setter soup	Every private field gets a public get/set pair	Expose behavior, not raw field access (Ch.1)
Deep inheritance chains	5+ levels of <code>: public</code>	Flatten with composition or interfaces
Inheritance for code reuse only	No real is-a, just wants shared methods	Composition (Ch.9)

BEST PRACTICES

Reach for a pattern because it names a problem you actually have, never to "look more design-pattern-y." SOLID and composition-over-inheritance are the two ideas that carry the most weight in a real code review.

MINI QUIZ

1. Explain, in your own words, why `Square : public Rectangle` with mutable width/height violates LSP.
2. What OOP mechanism does the Strategy pattern rely on?
3. Why is a static local variable inside a function a common, thread-safe way to implement a Singleton in C++11+?

14 Interview Q&A Bank

75+ questions: five deep-dive categories (definition, compare/contrast, output prediction, debugging/UB, design/refactor) with full answers, plus a rapid-fire FAQ round cross-referenced against seven interview-prep sources. Each deep-dive answer states what the interviewer is actually testing.

14.1 Definition questions

DEFINITION

D1. What is encapsulation, precisely?

Bundling data with the code that operates on it, and restricting outside access to that data, so the class itself can guarantee its own invariants. Not just "hiding data" — the payoff is invariant protection (Ch.1.5).

Tests: whether you can name the actual payoff, not just recite "hiding."

DEFINITION**D2. What is a pure virtual function?**

A virtual function declared with `= 0`, with no implementation required in that class. Any class with at least one is abstract and cannot be instantiated (Ch.4.2).

Tests: whether you know it can still have a body, callable explicitly as `Base::f()` — a common follow-up.

DEFINITION**D3. What is RAI?**

Resource Acquisition Is Initialization — tying a resource's lifetime to an object's lifetime: acquire in the constructor, release in the destructor, so the resource is released automatically no matter how the scope is exited (Ch.8.4).

Tests: whether you connect it to exception safety, not just "constructors and destructors."

DEFINITION**D4. What is the vtable?**

A per-class array of function pointers, one slot per virtual function, used to resolve virtual calls at runtime via each object's hidden vptr (Ch.7.2).

Tests: "per-class" vs "per-object" — a very common confusion to catch candidates on.

DEFINITION**D5. What is object slicing?**

Losing a derived object's extra data and vtable identity when it's copied into a base-typed value (e.g. passed by value into a base-typed parameter) — the copy only contains the base part (Ch.7.4).

Tests: can you name the fix (pass by reference/pointer) unprompted.

DEFINITION**D6. What is the Rule of 5?**

If a class needs to define any one of destructor, copy constructor, copy assignment, move constructor, or move assignment, it should generally define all five, because the compiler-generated defaults for the others are unlikely to be correct once one is customized (Ch.8.1).

Tests: whether you know Rule of 0 is actually the preferred default, not Rule of 5.

DEFINITION**D7. What does `explicit` do on a constructor?**

Prevents that constructor from being used for implicit conversions. Without it, a single-argument constructor lets the compiler silently convert a value of that argument's type into your class wherever your class type is expected — often surprising.

```
class Meters { public: Meters(double m); };  
void setDistance(Meters m);  
setDistance(5.0); // compiles WITHOUT explicit -- implicit double->Meters conversion  
// with 'explicit Meters(double m);', the line above is a compile error instead
```

Tests: whether you reach for `explicit` by default in your own code, not just define it.

DEFINITION**D8. What is CRTP?**

Curiously Recurring Template Pattern — a class template parameterized by its own derived class, used to achieve compile-time "polymorphism" with zero virtual-call overhead (Ch.11.2).

Tests: the specific trade-off (no runtime polymorphism / mixed-type containers) versus real virtual dispatch.

DEFINITION

D9. What is the PIMPL idiom?

Hiding a class's private implementation behind an opaque pointer to a forward-declared "Impl" class, so implementation changes don't force recompilation of every including file (Ch.11.3).

Tests: knowing the destructor must be defined where `Impl` is complete (the `.cpp` file).

DEFINITION

D10. What is `mutable` for?

Allowing a specific data member to be modified even from inside a `const` member function — reserved for implementation-detail state like caches, not for an object's logical value (Ch.3.5).

Tests: can you give a legitimate example (caching) and an abuse example unprompted.

14.2 Compare/contrast questions

COMPARE

C1. Overloading vs. overriding?

Overloading: multiple functions, same name, different parameter lists, resolved at compile time, unrelated to inheritance. Overriding: a derived class redefines a base `virtual` function with an identical signature, resolved at runtime via the vtable (Ch.6.3).

Tests: precise vocabulary — many candidates use these interchangeably.

COMPARE

C2. `struct` vs. `class` ?

Identical constructs except one default: `struct` members/inheritance default `public`, `class` defaults `private` (Ch.1.3).

Tests: knowing it's only the default, not a fundamental difference.

COMPARE

C3. Deep copy vs. shallow copy?

Shallow: copies a pointer member's value (the address) — both objects then share the same pointee. Deep: allocates new storage and copies the pointee's contents, giving each object independent ownership (Ch.8.2).

Tests: can you explain the double-free consequence of shallow-copying an owning pointer.

COMPARE

C4. Composition vs. inheritance — when do you pick each?

Inheritance when there's a genuine is-a relationship *and* you need polymorphic substitution through a base pointer/reference. Composition otherwise — especially for pure code reuse with no substitutability need (Ch.9.3).

Tests: whether "favor composition" is a reflex, not just a fact you can recite.

COMPARE

C5. `unique_ptr` vs. `shared_ptr` vs. `weak_ptr` ?

`unique_ptr` : sole, non-shared ownership, movable but not copyable, zero overhead versus a raw pointer. `shared_ptr` : reference-counted shared ownership, has atomic refcount overhead. `weak_ptr` : a non-owning observer of a `shared_ptr`-managed object, used to break reference cycles (Ch.8.5).

Tests: default choice — a strong candidate says " `unique_ptr` unless I have a specific reason to share."

COMPARE

C6. Static binding vs. dynamic binding?

Static: resolved by the compiler from the declared (static) type — non-virtual calls, overloads, hidden methods. Dynamic: resolved at runtime from the object's actual type, via the vtable — virtual calls through a pointer/reference (Ch.7.3).

Tests: can you say which one applies to a given code snippet on sight (often paired with an output-prediction question).

COMPARE

C7. Abstract class vs. interface, in C++ specifically?

C++ has no `interface` keyword. "Interface" is the idiom for an abstract class with only pure virtual functions and no data. A general "abstract class" may additionally have data and partially-implemented (non-pure) methods (Ch.4.4).

Tests: whether you know this is a convention, not a distinct language feature.

COMPARE

C8. Copy constructor vs. copy assignment operator — when does each run?

Copy constructor: when a *new* object is being created from an existing one (`T b = a;` , pass-by-value, return-by-value in some cases). Copy assignment: when an *already-existing* object is overwritten by another (`b = a;` where `b` already exists).

Tests: the "does the target already exist" distinction — trips up many candidates on `T b = a;` specifically (that's construction, not assignment, despite the `=` syntax).

COMPARE

C9. `const` member function vs. `const` object?

A `const` member function is a promise the *function* makes not to modify the object it's called on. A `const` object is a variable that can only call `const` member functions — the compiler enforces the pairing (Ch.3.4).

Tests: whether you understand why a non-const method simply won't compile on a const object.

COMPARE

C10. `virtual` inheritance vs. regular (multiple) inheritance?

Regular multiple inheritance from a shared base creates one base subobject per inheritance path (the diamond problem). `virtual` inheritance collapses those into exactly one shared subobject (Ch.9.1–9.2).

Tests: can you explain the memory-layout consequence, not just recite "it fixes the diamond."

C11. Interface inheritance vs. implementation inheritance?

Interface inheritance: inheriting a contract only (pure virtual, no shared behavior). Implementation inheritance: inheriting real behavior too, coupling the derived class to the base's internals (Ch.9.4).

Tests: whether you can say why interface inheritance is generally safer for polymorphism.

C12. `std::move` vs. an actual move?

`std::move` does nothing at runtime by itself — it's purely a cast to an rvalue reference, which makes the compiler *eligible* to select a move constructor/assignment overload if one exists. The actual "moving" (stealing internal resources) happens inside whatever move constructor/assignment gets selected.

Tests: a very common misconception — "does `std::move` move the object?" (No.)

14.3 Output-prediction questions

O1. What does this print?

```
class Base {
public:
    virtual void who() { std::cout << "Base\n"; }
};
class Derived : public Base {
public:
    void who() override { std::cout << "Derived\n"; }
};
int main() {
    Base b; Derived d;
    Base& ref = d;
    ref.who();
    b.who();
}
```

Answer: `Derived` then `Base`. `ref`'s static type is `Base&`, but its dynamic type is `Derived` — virtual dispatch resolves to `Derived::who()`. `b` really is a `Base`, so `b.who()` calls `Base::who()`.

Tests: basic dynamic dispatch through a reference.

OUTPUT

O2. What does this print?

```
class Base {
public:
    Base() { init(); }
    virtual void init() { std::cout << "Base::init\n"; }
};
class Derived : public Base {
public:
    void init() override { std::cout << "Derived::init\n"; }
};
int main() { Derived d; }
```

Answer: `Base::init` — *not* `Derived::init`. Inside `Base`'s constructor, the object's `vp`tr still points at `Base`'s vtable (the `Derived` part hasn't been constructed yet), so the "virtual" call resolves statically to `Base`'s own version. This is one of the most-missed output-prediction questions in C++ interviews.

Tests: whether you know virtual calls in constructors/destructors don't dispatch to derived overrides.

OUTPUT

O3. What does this print?

```
class Shape {
public:
    virtual double area() const { return 0.0; }
};
class Circle : public Shape {
public:
    double area() const override { return 100.0; }
};
void report(Shape s) { std::cout << s.area() << "\n"; }
int main() { Circle c; report(c); }
```

Answer: `0`. `report` takes `Shape` *by value*, so `c` is sliced down to its `Shape` part when passed in — `s` is a genuine `Shape`, not a `Circle`, so `s.area()` calls `Shape::area()`.

Tests: object slicing (Ch.7.4) recognized in running code, not just definition.

OUTPUT

O4. What does this print?

```
class A {
public:
    A() { std::cout << "A "; }
    ~A() { std::cout << "~A "; }
};
class B : public A {
public:
    B() { std::cout << "B "; }
    ~B() { std::cout << "~B "; }
};
int main() { B b; }
```

Answer: A B ~B ~A . Construction is base-first (A then B); destruction is the exact reverse (~B then ~A) (Ch.2.4).

Tests: construction/destruction order, a near-universal warm-up question.

OUTPUT

O5. What does this print, and is it well-defined?

```
class Base {
public:
    ~Base() { std::cout << "~Base\n"; }    // NOT virtual
};
class Derived : public Base {
public:
    ~Derived() { std::cout << "~Derived\n"; }
};
int main() {
    Base* b = new Derived();
    delete b;
}
```

Answer: Prints only ~Base — and this is technically **undefined behavior**, because delete -ing an object of derived type through a base pointer with a non-virtual destructor is UB per the standard (in practice, most compilers just skip ~Derived and run ~Base only, but the standard doesn't guarantee even that much).

Tests: not just "what prints" but "is this legal" — strong candidates flag the UB unprompted.

O6. What does this print?

```
struct Point { int x, y; };  
void modify(Point p) { p.x = 99; }  
int main() {  
    Point pt{1, 2};  
    modify(pt);  
    std::cout << pt.x;  
}
```

Answer: 1. `modify` takes `Point` by value — it modifies a local copy, not `pt` itself. No inheritance/virtual involved here; this is plain pass-by-value semantics, worth contrasting with O3's slicing scenario.

Tests: distinguishing plain by-value semantics from object slicing specifically.

14.4 Debugging / undefined-behavior identification

U1. Find the bug.

```
class Buffer {
public:
    explicit Buffer(size_t n) : size_(n), data_(new int[n]) {}
    ~Buffer() { delete[] data_; }
private:
    size_t size_;
    int* data_;
};

int main() {
    Buffer a(10);
    Buffer b = a;    // compiler-generated copy ctor: shallow copy
}    // both a and b destruct here
```

Bug: Double free. `Buffer` defines a destructor but not a copy constructor — the compiler generates a shallow one, so `a.data_ == b.data_`. When both go out of scope, `delete[]` runs twice on the same pointer: undefined behavior. **Fix:** write a deep-copying copy constructor (Rule of 3/5) or, better, replace the raw pointer with a `std::vector<int>` (Rule of 0).

Tests: recognizing the compiler-generated-shallow-copy trap on sight.

U2. Find the bug.

```
class Animal {  
public:  
    virtual void speak() const = 0;  
};  
class Dog : public Animal {  
public:  
    void speak() const override { std::cout << "Woof\n"; }  
};  
int main() {  
    Animal* a = new Dog();  
    delete a;    // Animal has no destructor at all, let alone virtual  
}
```

Bug: `Animal` has no explicit destructor — the compiler-generated one is non-virtual. Deleting a `Dog` through an `Animal*` is UB, same root cause as O5. **Fix:** `virtual ~Animal() = default;` .

Tests: the missing-virtual-destructor bug appearing without an explicit non-virtual destructor written — some candidates only catch it when they see `~Base(){...}` explicitly.

U3. Find the bug.

```
class Widget {
public:
    Widget& operator=(const Widget& other) {
        delete[] data_;
        data_ = new int[other.size_];
        std::copy(other.data_, other.data_ + other.size_, data_);
        size_ = other.size_;
        return *this;
    }
private:
    int* data_ = nullptr;
    size_t size_ = 0;
};

int main() {
    Widget w;
    w = w;    // self-assignment
}
```

Bug: No self-assignment guard. `w = w` deletes `data_`, then immediately reads `other.data_` — which is the same, now-dangling pointer. Use-after-free. **Fix:** add `if (this == &other) return *this;` at the top (Ch.10.2).

Tests: self-assignment as a deliberate edge case, not just "does copy assignment look right."

U4. Find the bug.

```
class Node {
public:
    std::shared_ptr<Node> next;
    std::shared_ptr<Node> prev;
};

int main() {
    auto a = std::make_shared<Node>();
    auto b = std::make_shared<Node>();
    a->next = b;
    b->prev = a;    // cycle
}    // a and b go out of scope here
```

Bug: Reference cycle. `a` and `b` hold `shared_ptr`s to each other; each keeps the other's refcount above zero forever, even after both local variables go out of scope. Neither `Node` is ever destroyed — a memory leak, not a crash. **Fix:** make one direction (typically `prev`) a `std::weak_ptr<Node>` instead, which observes without incrementing the refcount.

Tests: the single most common `shared_ptr` interview gotcha.

U5. Find the bug.

```
class Base {
public:
    virtual ~Base() = default;
    void greet() { std::cout << "Hi from "; name(); }
    void name() { std::cout << "Base\n"; }    // NOT virtual
};
class Derived : public Base {
public:
    void name() { std::cout << "Derived\n"; }    // hides, does not override -- no `virtual`, no
    `override`
};
int main() {
    Base* b = new Derived();
    b->greet();
}
```

Bug (or at least surprise): Not memory-unsafe, but almost certainly not what the author intended. `name()` is not `virtual`, so `Derived::name()` merely *hides* `Base::name()` rather than overriding it. `greet()` calls `name()` from inside `Base`, which statically binds to `Base::name()` — prints `"Base"`, not `"Derived"`. Adding `override` to `Derived::name()` without `virtual` on `Base::name()` would have caught this at compile time as an error.

Tests: hiding vs. overriding (Ch.5.4) in a realistic-looking snippet.

U6. Find the bug.

```
class Cache {
public:
    int get(int key) const {
        hits++; // COMPILER ERROR: can't modify hits_ in a const method
        return data_.at(key);
    }
private:
    int hits_ = 0;
    std::map<int,int> data_;
};
```

Bug: Won't compile — `get()` is `const` but modifies `hits_`. **Fix:** mark `hits_` as `mutable int hits_ = 0;` — a legitimate use, since hit-counting is bookkeeping, not part of the cache's logical externally-visible value (Ch.3.5).

Tests: recognizing a legitimate `mutable` use case, not just "add mutable to make it compile" without justification.

14.5 Design / refactor questions

R1. This class violates SRP. Refactor it.

```
class Report {
public:
    void addData(const std::string& d);
    void formatAsHtml();
    void saveToDisk(const std::string& path);
    void sendByEmail(const std::string& addr);
};
```

Answer: `Report` is doing data storage, formatting, persistence, *and* networking — four reasons to change. Split into `Report` (data only), `ReportFormatter` (formatting), `ReportStorage` (persistence), `ReportMailer` (email) — each depending on `Report`'s public interface, not on each other.

Tests: can you name which specific responsibilities are tangled, not just "it's too big."

DESIGN

R2. Redesign this to remove the `dynamic_cast` chain.

```
void render(Shape* s) {  
    if (auto* c = dynamic_cast<Circle*>(s)) { /* draw circle */ }  
    else if (auto* r = dynamic_cast<Rectangle*>(s)) { /* draw rectangle */ }  
}
```

Answer: Add a pure virtual `draw()` to `Shape`, implement it in each concrete subclass, and replace the whole chain with `s->draw();`. This is exactly the "missing virtual function" smell from Ch.7.5 — every new shape type would otherwise require finding and editing every `dynamic_cast` chain in the codebase (violates Open/Closed too).

Tests: connects a concrete code smell to two named principles (Ch.7 RTTI guidance + Open/Closed).

DESIGN

R3. Design a `Shape` hierarchy that supports adding new shapes AND new operations (area, draw, serialize) without editing existing shape classes each time. What's the tension?

Answer: This is the classic "expression problem." Virtual functions make adding new *shapes* easy (just derive and override) but adding a new *operation* requires touching every existing shape class to add the new virtual method. The reverse (a free function per operation, switching on type) makes new operations easy but new shapes hard. For a fixed, rarely-changing set of operations with a growing set of shapes, virtual functions win — which is the common case, and why OOP defaults to it. If operations grow faster than types, consider the Visitor pattern as a middle ground (worth mentioning, out of scope to implement here).

Tests: awareness that OOP's virtual-dispatch approach has a real trade-off, not just "always use virtual functions."

DESIGN

R4. A colleague proposes making every data member `protected` "so derived classes can access what they need." What's your feedback?

Answer: Push back. `protected` data means every derived class — including ones written later, by other people — has unrestricted read/write access to the base's internal state, silently coupling every subclass to the base's implementation details forever. Prefer `private` data with a minimal `protected` (or public) set of methods that express what a derived class is actually allowed to do — that's still an interface, just one aimed at subclasses instead of external callers.

Tests: whether you'll defend encapsulation even against a "convenience" argument — a common senior-level judgment question.

DESIGN

R5. Refactor this to follow the Rule of 0.

```
class Team {
public:
    Team(int n) : members_(new std::string[n]), count_(n) {}
    ~Team() { delete[] members_; }
    Team(const Team& o) : members_(new std::string[o.count_]), count_(o.count_) {
        std::copy(o.members_, o.members_ + o.count_, members_);
    }
    Team& operator=(const Team& o) { /* ...similar boilerplate... */ }
private:
    std::string* members_;
    int count_;
};
```

Answer: Replace the raw `std::string*` array with `std::vector<std::string>`. `vector` already correctly implements deep copy, move, and cleanup — deleting all five special member functions from `Team` and letting the compiler generate correct defaults (Rule of 0).

Tests: recognizing that most "I need Rule of 5" situations are actually "I'm using the wrong container/type."

R6. When would you choose CRTP over a virtual interface for a plugin-style system?

Answer: Only when (a) the set of concrete types is known and fixed at compile time (not loaded as plugins at runtime — CRTP fundamentally can't do that, since each instantiation is a distinct type), and (b) profiling has shown virtual dispatch overhead actually matters (e.g. an extremely hot per-element loop). A genuine plugin system, where types are discovered/loaded at runtime, requires real runtime polymorphism — a virtual interface is the only correct choice there.

Tests: whether you understand CRTP's fixed-at-compile-time limitation well enough to rule it out for the wrong use case, not just cite it as "the fast alternative."

14.6 Four more classics

Pulled in from a second cross-referenced pass — these four come up often enough to deserve full answers, not just a rapid-fire line.

D11. Why can't constructors be virtual?

Virtual dispatch requires an already-set-up vptr pointing at a vtable — but the constructor is precisely what sets the vptr up in the first place (Ch.7.2). There is no valid vptr yet to dispatch through before construction begins, so "a virtual constructor" is a contradiction: dynamic dispatch needs a live object to dispatch *on*, and construction is what brings the object into existence.

Tests: whether you can connect this back to the vptr/vtable mechanics, not just state it as a rule to memorize.

ADVANCED

D12. What is copy elision / RVO, and what changed in C++17?

The compiler constructs a returned object directly in the caller's storage, skipping an intermediate copy/move entirely. In **C++17, for prvalues** (e.g. `return Widget(args);`) this is **mandatory** — guaranteed by the standard, not merely permitted — so the copy/move constructor doesn't even need to be accessible. In C++11/14 it was only an optional-but-common optimization under the "as-if" rule, so a copy/move constructor still had to exist (even if never actually called).

Tests: the specific mandatory-in-C++17 vs. permitted-earlier distinction — a strong signal of genuinely current standard knowledge, not memorized-in-2012 knowledge.

DEFINITION

D13. When is `const_cast` legitimate?

Almost exclusively to call a legacy or third-party API that fails to mark a parameter `const` despite not modifying it, when you can't change that API's signature. Using it to actually mutate an object that was originally declared `const` (as opposed to one that's merely accessed through a `const` pointer/reference to a non-const object) is undefined behavior.

Tests: whether you know the difference between "object is const" (mutating via `const_cast` is UB) and "reference/pointer is const but object isn't" (mutating via `const_cast` is legal, if occasionally in poor taste).

DEBUG/DESIGN

D14. Why can't you call a derived-only method through a base pointer, and what's the right fix?

```
class Animal { public: virtual ~Animal() = default; };
class Dog : public Animal { public: void fetch() {} };
Animal* a = new Dog();
// a->fetch(); -- COMPILE ERROR: Animal has no fetch()
```

The compiler only knows `a`'s *static* type, `Animal*` — it has no compile-time guarantee the object pointed to is actually a `Dog`, so it refuses to compile a call to a method it can't see declared on `Animal`. Fix with a checked `dynamic_cast<Dog*>(a)` (Ch.7.5) — or, better, ask whether `fetch()` is common enough behavior that it belongs on the base interface instead.

Tests: connects a basic compile error to the design question it's really asking (Ch.7.5's "missing virtual function" smell).

14.7 Rapid-fire FAQ round

75 questions, cross-referenced against GeeksforGeeks, InterviewBit, Simplilearn, Edureka, MyGreatLearning, ScienceTechEasy, and FinalRoundAI, collapsed here to the ones not already answered in full above. Frequently Asked = appeared on 3+ sources; C++ Specific = a C++-only conceptual trap.

What is OOP, in one line? FREQUENTLY ASKED

A paradigm organizing code around objects — data + the behavior on that data — instead of functions acting on separate data, built on encapsulation, abstraction, inheritance, polymorphism.

Name the four pillars, fast. FREQUENTLY ASKED

Encapsulation, abstraction, inheritance, polymorphism (Ch.1, 4, 5, 6).

Advantages of OOP? FREQUENTLY ASKED

Modularity, reusability (inheritance/composition), maintainability (encapsulation limits ripple effects), flexibility (polymorphism), intuitive real-world modeling.

Is C++ a "purely" OOP language?

No — multi-paradigm. C++ allows free functions and globals outside any class, unlike languages where everything must live in a class.

Name the five inheritance "shapes" C++ supports.

Single, multiple, multilevel (a chain), hierarchical (many derived from one base), hybrid (a combination — where diamonds can arise).

OOP vs. procedural programming?

FREQUENTLY ASKED

Procedural: functions operate on shared/global data, reuse via function calls. OOP: objects bundle data+behavior, encapsulate access, reuse via inheritance/composition.

Abstraction vs. encapsulation — the one-line distinction? FREQUENTLY ASKED

Abstraction hides complexity (the "how"); encapsulation hides data (restricts access). Encapsulation is often the mechanism that achieves abstraction.

Disadvantages of OOP?

Overkill for simple linear scripts; deep hierarchies are fragile ("fragile base class"); vtable/object overhead vs. plain procedural code; inheritance overused where composition would fit better.

Can a destructor be overloaded?

No — a destructor takes no parameters and a class has exactly one, since the compiler needs one unambiguous cleanup path regardless of how the object was built.

Multiple vs. multilevel inheritance?

Multiple: one class inherits directly from 2+ unrelated bases at the same level. Multilevel: a linear chain across generations (C from B, B from A).

Limitations of inheritance, beyond the diamond problem? FREQUENTLY ASKED

Tight base/derived coupling ("fragile base class"), hard-to-trace deep hierarchies, and reaching for it for reuse alone with no genuine is-a (Ch.9's core warning).

Can a derived class touch its base's private members?

No — private is visible only within the declaring class itself, never derived classes. Use protected if derived classes legitimately need direct access.

What is function/name hiding, and how do you undo it? C++ SPECIFIC

A derived function sharing a base function's name hides *all* base overloads of that name for derived-typed access — not just the one it resembles. A `using Base::name;` declaration in the derived class brings the base overloads back into scope.

What does `::` (scope resolution) do?

Qualifies a name with its scope — defining a member out-of-line (`void C::f(){}`), reaching a static member (`C::count()`), or explicitly calling a hidden/overridden base version (`Base::f()`).

Operator overloading vs. function overloading?

Both are compile-time polymorphism resolved by argument types, but operator overloading specifically redefines what an existing operator symbol does for a type (`operator+`), rather than defining multiple same-named functions.

Association vs. aggregation vs. composition, with an example each? FREQUENTLY ASKED

Association: general "uses" link, independent lifetimes (a Teacher and a Student). Aggregation: weak "has-a," part can outlive the whole (a Department and its Professors). Composition: strong "has-a," part dies with the whole (a Car and its Engine) — Ch.9.3, Ch.13.3.

Virtual function vs. pure virtual function, precisely?

Virtual: has a (possibly default) implementation derived classes may override. Pure virtual (`= 0`): no implementation in that class; derived classes must override it to be instantiable (Ch.4.2, Ch.6.2).

Can a private virtual function be overridden? C++ SPECIFIC

Yes, surprisingly — overriding only requires the function be visible at the point of *declaration* in the base, not at the call site. A common trick question, since static/private-in-the-non-virtual-sense functions truly cannot be overridden.

What storage classes exist in C++ (beyond access specifiers)?

`auto` (default local, scope-lifetime), `static` (program-lifetime / shared per class), `extern` (defined in another translation unit), `mutable` (modifiable even in a const method, Ch.3.5).

Which operators can't be overloaded? FREQUENTLY ASKED

A fixed small set: `::`, `.`, `.*`, `?:`, and `sizeof` — overloading these would undermine guarantees the language depends on.

Prefix vs. postfix `operator++` ?

Distinguished by a dummy `int` parameter on postfix. Prefix returns a reference to the already-incremented object; postfix must save the old value, increment, and return the old value by copy — postfix is inherently more expensive.

Does C++ have garbage collection, unlike Java?

No — the programmer (or RAII/smart pointers) reclaims heap memory. Java's GC is automatic but non-deterministic (finalize isn't guaranteed to run promptly); a C++ destructor runs deterministically and immediately.

Why is Singleton sometimes called an anti-pattern?

It introduces global mutable state — harder to unit-test (can't substitute a mock instance), creates hidden cross-module dependencies, and risks subtle bugs under concurrent access if synchronization isn't careful. Dependency injection is often the more testable alternative.

Stack vs. heap allocation — the trade-off?

Stack: automatic cleanup at scope end, fast, but fixed lifetime. Heap: flexible lifetime (can outlive the creating scope — needed for polymorphic objects via base pointers), but slower and needs explicit or smart-pointer-managed cleanup.

What is a dangling pointer?

A pointer to memory that's already been freed (after `delete`, or after a stack object's scope ends) — dereferencing it is undefined behavior.

What are coupling and cohesion?

Coupling: how dependent classes are on each other's internals (lower is better). Cohesion: how focused a single class's responsibilities are (higher is better). Good OOP design: low coupling, high cohesion.

15 Practice Problems & Mini Designs

Short exercises first (with solutions), then five complete mini designs — each a different pattern (Strategy, Template Method, Observer+Decorator, a plain interface, Composite) — with progressively harder extensions. Try each before reading the solution.

15.1 Warm-up exercises

EXERCISE 1

Write a `Temperature` class that stores Celsius internally, with an invariant that it never goes below absolute zero (-273.15°C).

```
class Temperature {
public:
    explicit Temperature(double celsius) : celsius_(std::max(celsius, -273.15)) {}
    double celsius() const { return celsius_; }
    void setCelsius(double c) { celsius_ = std::max(c, -273.15); }
private:
    double celsius_;
};
```

Why it matters: exercises Ch.1's invariant-protection idea — every entry point that changes state enforces the rule, not just the constructor.

EXERCISE 2

Add a virtual destructor to a class hierarchy that's missing one, and explain exactly which line was unsafe before your fix.

Solution: Add `virtual ~Base() = default;` to the base. The unsafe line is any `delete basePtr;` where `basePtr`'s dynamic type is a derived class — before the fix, that line is undefined behavior (Ch.6.5).

EXERCISE 3

Convert a class using raw `new` / `delete` to use `unique_ptr` and Rule of 0.

Solution approach: Replace the raw pointer member with `std::unique_ptr<T>`, construct it with `std::make_unique<T>(...)`, and delete the destructor, copy constructor, and copy assignment operator entirely (or explicitly `= delete` the copy operations if the class should be move-only) — the compiler-generated destructor now correctly calls the `unique_ptr`'s destructor automatically.

EXERCISE 4

Write a class where `operator==` is correct but `operator!=`, if hand-written separately, could accidentally become inconsistent. Show the safe pattern.

```
class Point {
public:
    bool operator==(const Point& o) const { return x_ == o.x_ && y_ == o.y_; }
    bool operator!=(const Point& o) const { return !(*this == o); }    // defined IN TERMS OF ==, can't
    drift
private:
    int x_, y_;
};
```

EXERCISE 5

Given a base class `Employee` and derived `Manager`, write a function that prints a list of mixed employees polymorphically without slicing.

```
void printAll(const std::vector<std::unique_ptr<Employee>>& team) {
    for (const auto& e : team) std::cout << e->describe() << "\n";    // virtual call, no slicing
}
```

15.2 Mini design 1: Parking Spot Pricing (Strategy)

Requirements

A parking garage charges different rates depending on vehicle type (motorcycle/car/bus) and needs to support adding new pricing strategies (e.g. a weekend discount) without modifying existing vehicle classes.

```

class PricingStrategy {
public:
    virtual double hourlyRate() const = 0;
    virtual ~PricingStrategy() = default;
};
class StandardRate : public PricingStrategy {
public:
    explicit StandardRate(double rate) : rate_(rate) {}
    double hourlyRate() const override { return rate_; }
private:
    double rate_;
};
class ParkingSpot {
public:
    ParkingSpot(int id, std::unique_ptr<PricingStrategy> pricing)
        : id_(id), pricing_(std::move(pricing)) {}
    double cost(double hours) const { return pricing_>hourlyRate() * hours; }
private:
    int id_;
    std::unique_ptr<PricingStrategy> pricing_;    // composition: pricing is swappable
};

```

Extension 1 (easier): Add a `WeekendDiscountRate` decorator that wraps another `PricingStrategy` and applies a 20% discount — no changes to `ParkingSpot` needed.

Extension 2 (harder): Add a `SurgePricing` strategy that adjusts rate based on current occupancy, requiring the strategy to query the garage's live state — discuss how you'd pass that dependency in without creating a circular include (hint: an interface the garage implements, passed by reference).

15.3 Mini design 2: Document Export System (Template Method + Composition)

Requirements

Export a `Document` to multiple formats (PDF, HTML, plain text) that share a common three-step process (open, write body, close) but differ in exactly how each step is done.

```

class Exporter {
public:
    void exportDoc(const std::string& content) {    // Template Method: fixed skeleton
        open();
        writeBody(content);
        close();
    }
    virtual ~Exporter() = default;
protected:
    virtual void open() = 0;
    virtual void writeBody(const std::string& content) = 0;
    virtual void close() = 0;
};

class HtmlExporter : public Exporter {
protected:
    void open() override { std::cout << "<html><body>"; }
    void writeBody(const std::string& c) override { std::cout << c; }
    void close() override { std::cout << "</body></html>"; }
};

```

This is the **Template Method** pattern: `exportDoc()` is non-virtual (the fixed algorithm shape), while the three steps it calls are pure virtual (the customizable parts) — note the steps are `protected`, since only `Exporter` itself should call them directly.

Extension 1 (easier): Add a `PdfExporter`.

Extension 2 (harder): Add an optional `addWatermark()` step that only some formats support — without forcing every exporter to implement a no-op. (Hint: give it a non-pure virtual default that does nothing, which subclasses may optionally override — this is a legitimate, narrow case for a non-pure virtual function in an otherwise-abstract class.)

15.4 Mini design 3: Event Notification System (Observer + Decorator, combined)

Requirements

Multiple parts of a system need to react when an order ships, without the shipping code knowing about them directly. Separately, notifications sometimes need retry-on-failure behavior layered on.

```

class Observer {
public:
    virtual void onShipped(const std::string& orderId) = 0;
    virtual ~Observer() = default;
};

class OrderShipping {
public:
    void addObserver(Observer* o) { observers_.push_back(o); }
    void ship(const std::string& orderId) {
        // ... shipping logic ...
        for (auto* o : observers_) o->onShipped(orderId); // notify everyone, decoupled
    }
private:
    std::vector<Observer*> observers_; // non-owning -- OrderShipping does not own its observers
};

// Decorator, from Ch.13's pattern tour, layered on top of one observer:
class RetryingObserver : public Observer {
public:
    RetryingObserver(std::unique_ptr<Observer> inner, int maxAttempts)
        : inner_(std::move(inner)), maxAttempts_(maxAttempts) {}
    void onShipped(const std::string& orderId) override {
        for (int i = 0; i < maxAttempts_; ++i) {
            try { inner_->onShipped(orderId); return; } catch (...) { /* retry */ }
        }
    }
private:
    std::unique_ptr<Observer> inner_;
    int maxAttempts_;
};

```

Extension 1 (easier): Add an `EmailObserver` and an `SmsObserver`.

Extension 2 (harder): `OrderShipping` stores raw `Observer*` — discuss the lifetime risk (a dangling pointer if an observer is destroyed before `OrderShipping`), and redesign using `std::weak_ptr<Observer>` observers registered from `shared_ptr`-owned objects, skipping any that have expired at notify time.

15.5 Mini design 4: Pluggable Logger (interface + composition)

Requirements

Log messages to a destination that can be swapped without changing calling code — console today, a file or network sink tomorrow, with no edits to `Logger` itself.

```

class LogSink {
public:
    virtual void write(const std::string& msg) = 0;
    virtual ~LogSink() = default;
};
class ConsoleSink : public LogSink {
public:
    void write(const std::string& msg) override { std::cout << msg << "\n"; }
};
class Logger {
public:
    explicit Logger(std::unique_ptr<LogSink> sink) : sink_(std::move(sink)) {}
    void log(const std::string& msg) { sink_->write(msg); }
private:
    std::unique_ptr<LogSink> sink_; // composition + polymorphism (Ch.9.3)
};

```

Extension 1 (easier): Add a `FileSink` that writes to a `std::ofstream`.

Extension 2 (harder): Support multiple simultaneous sinks by holding `std::vector<std::unique_ptr<LogSink>>` and writing to each in `log()` — discuss what should happen if one sink throws mid-loop (does logging to the remaining sinks still matter?).

15.6 Mini design 5: Shape Hierarchy with Composite

Requirements

Model shapes polymorphically (area, perimeter, printing), then support *groups* of shapes that behave like a single shape themselves — a group's area is the sum of its members', recursively.

```

class Shape {
public:
    virtual double area() const = 0;
    virtual ~Shape() = default;
};
class Circle : public Shape {
public:
    explicit Circle(double r) : r_(r) {}
    double area() const override { return 3.14159 * r_ * r_; }
private:
    double r_;
};
class CompositeShape : public Shape { // a group of shapes that IS ITSELF a Shape
public:
    void add(std::unique_ptr<Shape> s) { children_.push_back(std::move(s)); }
    double area() const override {
        double total = 0;
        for (const auto& c : children_) total += c->area(); // recurses into nested composites too
        return total;
    }
private:
    std::vector<std::unique_ptr<Shape>> children_;
};

```

This is the **Composite** pattern: because `CompositeShape` is itself a `Shape`, client code that calls `shape->area()` doesn't need to know or care whether it's holding one circle or a deeply nested group of a thousand — the recursion is invisible from outside.

Extension 1 (easier): Add a `Rectangle` and nest a `CompositeShape` inside another `CompositeShape`.

Extension 2 (harder): Add a `remove(Shape*)` to `CompositeShape` — discuss why searching by raw pointer identity through a `vector<unique_ptr<Shape>>` is awkward, and what it would take to support it cleanly (hint: this is a genuine, common ownership-design tension worth being able to discuss, not a trick with one clean answer).

HOW TO PRACTICE THESE

Implement each from the requirements alone before looking at the code. Then do the extensions the same way. The "harder" extensions are deliberately open-ended — there's more than one reasonable answer; the point is reasoning about trade-offs, exactly like a real interview.

16 Revision: Cheat Sheets, Glossary & Checklist

Everything above, compressed to what you actually need the morning of the interview.

16.1 The four pillars, compressed

- **Encapsulation** — bundle data + behavior; protect invariants via access control, not just "hide data."
- **Abstraction** — expose an interface, hide the how (abstract classes, pure virtual functions).
- **Inheritance** — is-a; `public` by default; favor composition unless you need polymorphic substitution.
- **Polymorphism** — runtime (virtual/vtable) vs. compile-time (overloading/templates/CRTP); "polymorphism" unqualified usually means runtime.

16.2 Internals, compressed

One vptr per *object*, one vtable per *class* · virtual call = object → vptr → vtable slot → indirect call · pass-by-value into a base-typed parameter slices · missing virtual destructor + `delete` via base pointer = UB · virtual calls inside constructors/destructors resolve statically, to the currently-under-construction class, never to a not-yet-built derived override.

16.3 Rule of 0/3/5, compressed

Prefer Rule of 0 (let members manage themselves — `string`, `vector`, smart pointers) · Rule of 3 = destructor + copy ctor + copy assign, for raw-resource owners · Rule of 5 adds move ctor + move assign, both `noexcept` · RAll ties resource lifetime to object lifetime.

16.4 All comparison tables, in one place

TOPIC	OPTION A	OPTION B
Polymorphism	Compile-time: zero overhead	Runtime: vtable indirection
Ownership	<code>unique_ptr</code> : sole owner	<code>shared_ptr</code> : ref-counted
Relationship	Composition: owns, same lifetime	Aggregation: has, independent lifetime
Inheritance type	Interface: contract only	Implementation: behavior too
Binding	Static: compile-time, declared type	Dynamic: runtime, actual type

16.5 Glossary

Vtable

Per-class array of virtual function pointers.

Vptr

Per-object hidden pointer to its class's vtable.

Slicing

Losing derived-specific data/behavior when copying into a base-typed value.

RAII

Resource Acquisition Is Initialization — tying resource lifetime to object lifetime.

RTTI

Runtime Type Information — `dynamic_cast`, `typeid`.

CRTP

Curiously Recurring Template Pattern — compile-time polymorphism via templates.

PIMPL

Pointer to Implementation — hides private details behind an opaque pointer.

EBO

Empty Base Optimization — zero-size empty base subobjects.

Invariant

A condition a class guarantees always holds for its objects.

LSP

Liskov Substitution Principle — a derived object must be usable anywhere a base is expected without surprise.

Type erasure

Hiding a concrete type behind a uniform runtime interface without requiring shared inheritance.

Move semantics

Transferring a resource's ownership instead of duplicating it.

16.6 Day-before-the-interview checklist

```
[ ] Can you trace, from memory, why a missing virtual destructor is UB?  
[ ] Can you draw the vptr/vtable picture for a 2-level hierarchy on a whiteboard?  
[ ] Can you explain object slicing AND show the fix, unprompted?  
[ ] Can you write a correct Rule-of-5 class from scratch, including noexcept on moves?  
[ ] Can you explain a shared_ptr cycle and its weak_ptr fix?  
[ ] Can you explain why a virtual call inside a constructor doesn't dispatch to derived?  
[ ] Can you state composition-vs-inheritance guidance with a concrete example?  
[ ] Can you spot a Liskov Substitution violation in a Square/Rectangle-style example?  
[ ] Have you redone every chapter's mini quiz without looking at the chapter?  
[ ] Have you implemented at least one Ch.15 mini design from requirements alone?
```

How to Convert This Guide to PDF

This page is already print-formatted — code blocks wrap instead of clipping, tables resize to the page, and the contents index carries real page numbers once exported.

1. Click the **Save as PDF** button in the top bar (uses your browser's native print engine), or press `Ctrl/Cmd+P`.
2. In the print dialog, set **Destination** to *Save as PDF*, and enable **Background graphics** so the callout boxes and diagrams keep their color.
3. Margins: *Default* or *None* both work well with this layout.

The Definitive C++ OOP Handbook · combines and deduplicates three prior guides into one: the C++ OOP Mastery Handbook's full 16-chapter course, the C++ OOP Handbook's copy-elision/const_cast/base-pointer deep dives, and a 75-question cross-referenced FAQ bank · targets C++17